

AGENT EXPERIMENTS

Personal Productivity & Task Execution Agent — 6 experiments

Prepared by: Hooria · AI Summer Internship — NLP & AI Agents Track · July 2026

Methodology

Each experiment below is implemented as a real, runnable script under `app/evaluation/experiments/` that exercises the live agent — real LLM calls, real tools, real database — rather than being reasoned about in the abstract. Results columns are marked “Pending — run experiment” because they require a live `GEMINI_API_KEY` (and optionally `OPENAI_API_KEY` for Experiment 6) that this environment does not have network access to call; fabricating plausible-looking numbers would defeat the purpose of an experiment. Run each script (or all of them via `run_all.py`) and the console output plus `app/evaluation/experiments/results/*.json` will contain the real figures to drop into the Result column below.

```
python -m app.evaluation.experiments.run_all
```

Full methodology detail (exact probes, transcripts, and formulas) is documented in `docs/experiments.md` alongside the code.

Experiment 1: Tool Description Quality

Hypothesis

Terse, one-line tool descriptions cause the model to confuse tools that are similar in surface form (e.g. `create_task` vs `create_reminder`, `complete_task` vs `update_task`). Detailed descriptions that explicitly name the disambiguation should raise tool selection accuracy specifically on those confusable requests.

Method

The same 13 tool schemas are built twice — once with short descriptions, once with detailed disambiguating descriptions — and run against 6 deliberately ambiguous probe requests (one per confusable pair) with everything else (system prompt, model, temperature) held constant.

Script: `app/evaluation/experiments/exp1_tool_descriptions.py`

Variables Measured & Results

Variable measured	How it's computed	Result
Tool selection accuracy — short descriptions	% of 6 probes where the expected tool was called, using terse descriptions	Pending — run experiment
Tool selection accuracy — detailed descriptions	% of 6 probes where the expected tool was called, using detailed descriptions	Pending — run experiment
Delta	Detailed accuracy – short accuracy, in percentage points	Pending — run experiment

Experiment 2: Structured versus Unstructured Output

Hypothesis

Asking for a fixed JSON schema produces far fewer downstream parsing failures than asking for the same information in free prose and parsing it afterward.

Method

5 meeting-note samples are each run through (a) the real generate_json structured path used by extract_meeting_actions, and (b) a free-text prompt requesting the same information in prose, which is then passed through a best-effort downstream parser (regex for an embedded JSON block, falling back to heading detection) to simulate what a naive integration would have to do.

Script: app/evaluation/experiments/exp2_structured_output.py

Variables Measured & Results

Variable measured	How it's computed	Result
Structured parsing failure rate	% of 5 samples where the JSON-schema response did not parse into the expected fields	Pending — run experiment
Unstructured parsing failure rate	% of 5 samples where the free-text response could not be recovered into the same schema	Pending — run experiment

Experiment 3: Model Temperature

Hypothesis

Higher temperature reduces tool-selection consistency and increases hallucination in structured extraction, at some cost/benefit to response variety.

Method

At temperatures 0.0, 0.5, and 1.0, runs 5 repetitions each of a tool-selection request and a meeting-notes extraction request (same transcript every time, with a known ground truth of which names/dates it actually contains).

Script: app/evaluation/experiments/exp3_temperature.py

Variables Measured & Results

Variable measured	How it's computed	Result
Tool selection consistency	% agreement with the modal (most common) tool pick across 5 reps, per temperature	Pending — run experiment
Tool selection accuracy	% of reps that picked the documented correct tool, per temperature	Pending — run experiment
Hallucination score	Fraction of extracted owners/decisions that do not trace back to the source transcript, per temperature	Pending — run experiment
Response quality (proxy)	Average summary length in characters, per temperature	Pending — run experiment

Full per-repetition detail (every tool pick, every extraction) is in exp3_temperature.json.

Experiment 4: Approval Prompt Design

Hypothesis

The agent pauses for explicit human approval before every write action, including when the request is phrased to explicitly discourage confirmation (e.g. “don't ask me, I'm sure”), because the gate is enforced in code rather than only via prompt wording.

Method

6 write-action requests, including two adversarial phrasings, are each run 3 times. Each run is checked for whether an approval pause occurred before the gated tool executed, and whether the agent's response ever claimed the action was done without a tool actually having run.

Script: `app/evaluation/experiments/exp4_approval_consistency.py`

Variables Measured & Results

Variable measured	How it's computed	Result
Approval pause rate	% of 18 runs where the agent paused for approval before the gated tool ran (target: 100%)	Pending — run experiment
False-completion-claim rate	% of runs where the agent's text claimed an action happened without a corresponding tool call (target: 0%)	Pending — run experiment

Experiment 5: Maximum Agent Steps

Hypothesis

Very low step ceilings cut off genuinely multi-step workflows before they complete; very high ceilings add latency and LLM-call cost with diminishing completion gains once a workflow's natural step count is exceeded.

Method

3 requests that genuinely require 2–3 sequential tool calls are each run 3 times at step ceilings of 2, 4, 8, and 16 (MAX_AGENT_STEPS temporarily overridden per run).

Script: `app/evaluation/experiments/exp5_max_steps.py`

Variables Measured & Results

Variable measured	How it's computed	Result
Completion rate	% of runs that finished without hitting the step ceiling, per limit	Pending — run experiment
Looping incidents	Count of runs where the duplicate-call/loop guard fired, per limit	Pending — run experiment
Average latency	Mean wall-clock duration per run, per limit	Pending — run experiment
Average LLM calls (cost proxy)	Mean number of LLM calls per run, per limit — used since token-level cost isn't exposed uniformly across providers	Pending — run experiment

Experiment 6 (optional): Model Comparison

Hypothesis

Gemini 2.5 Flash and an OpenAI model achieve broadly comparable tool-selection accuracy on the same request set, with latency being the more likely differentiator.

Method

The same 8-case subset of the Assignment 5 evaluation dataset (single-tool and multi-tool categories) is run once against each configured provider, swapping LLM_PROVIDER between runs.

Script: `app/evaluation/experiments/exp6_model_comparison.py`

Variables Measured & Results

Variable measured	How it's computed	Result
Tool selection accuracy — Gemini	% of 8 cases correctly handled by gemini-2.5-flash	Pending — run experiment
Tool selection accuracy — OpenAI	% of 8 cases correctly handled by the configured OpenAI model	Pending — run experiment
Average latency — both providers	Mean run duration per provider	Pending — run experiment

Skipped automatically for any provider whose API key is not configured.